

Ingeniería del Software

Tema 8. Economía

TABLA DE CONTENIDOS

- 01** **El triángulo de la gestión**
- 02** **La inversión**
- 03** **Matriz Valor vs Coste**
- 04** **Leyes Heurísticas**

El triángulo de la gestión

El triángulo de la gestión





El proyecto se está retrasando, parece que no se van a cumplir las fechas comprometidas. ¿Qué podemos hacer?

- A) **Construir más desprisa.** Para ello dejaremos de hacer pruebas.
- B) **Incorporar más programadores al equipo.** Al pagar a más desarrolladores, reducimos el beneficio esperado.
- C) **Acotar el alcance funcional del producto.** Entregaremos un producto software con menos funcionalidades, ¡seguro que al cliente no le importa!

El triángulo de la gestión

“Lo que un programador hace en un mes, dos programadores lo pueden hacer en dos meses”

- Fred Brooks, *Ley de Brooks*

Brooks señala que incluir más trabajadores en los proyectos de software aumenta la complejidad de la gestión y hace crecer geométricamente los flujos de comunicación necesarios para el traspaso de conocimiento.

Añadir más programadores a un proyecto no adelantará su finalización, probablemente lo retrase aún más.

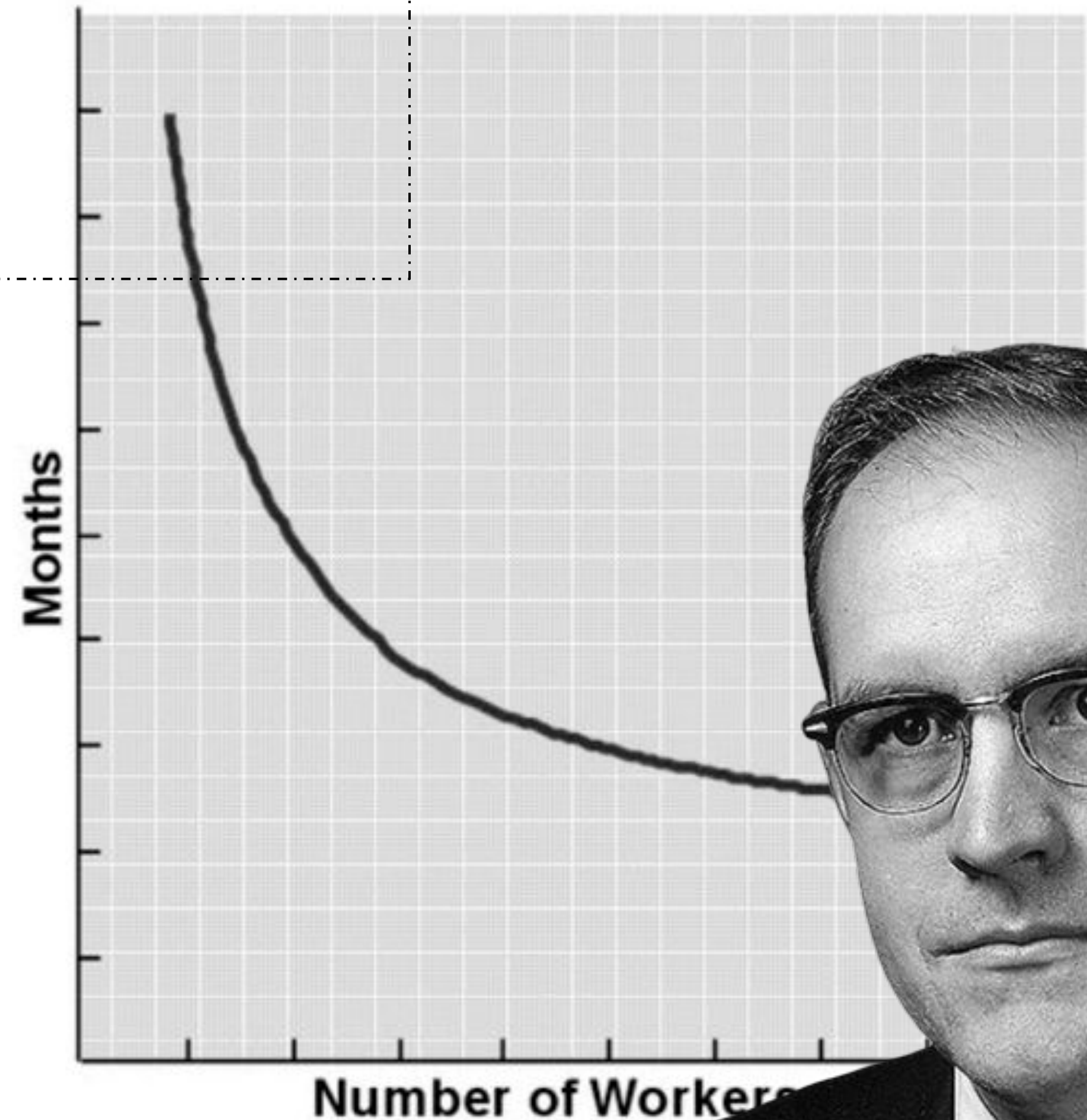


Fig. 2.3 Time versus number of workers for a partitionable task requiring

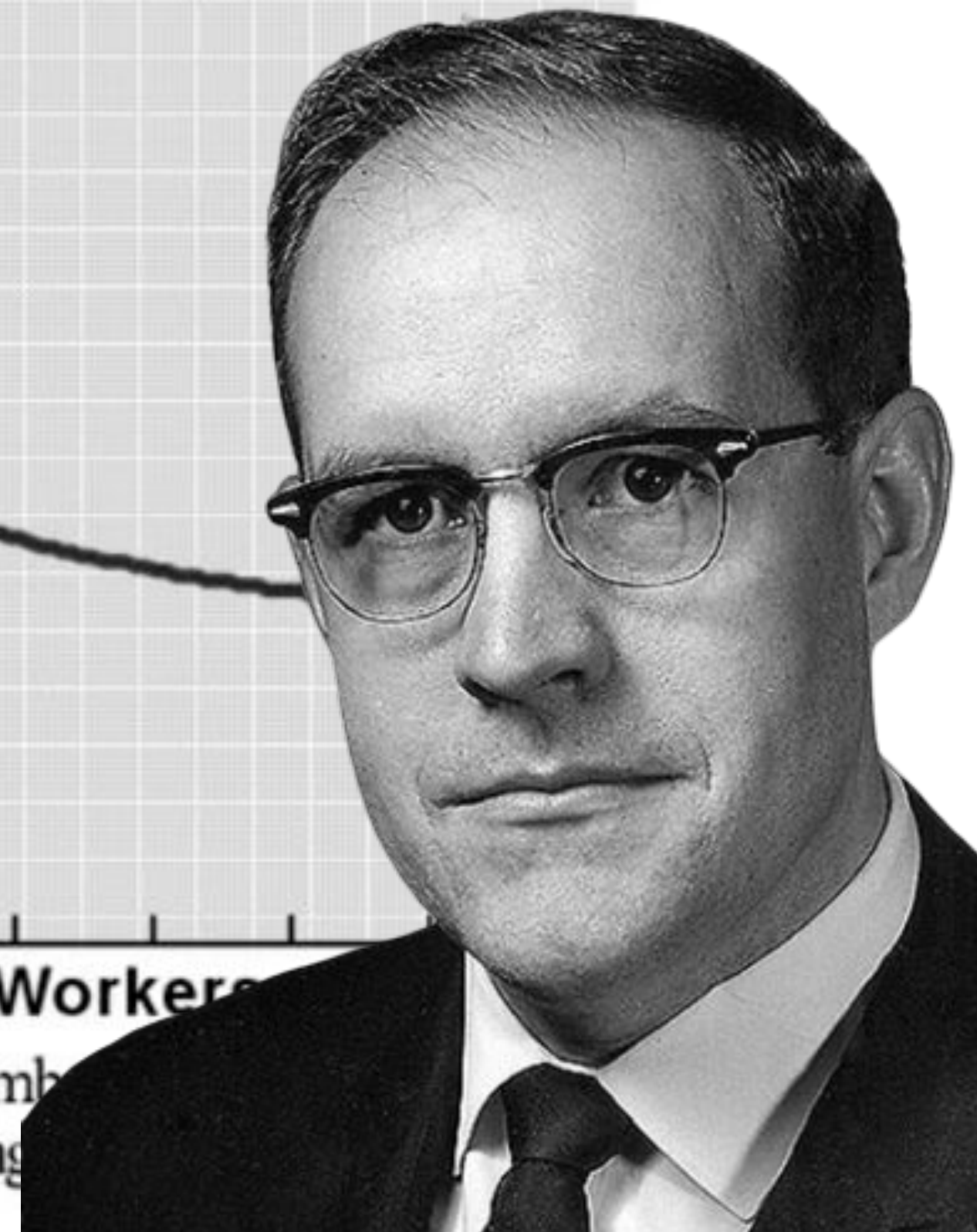


TABLA DE CONTENIDOS

- 01** El triángulo de la gestión
- 02** **La inversión**
- 03** Matriz Valor vs Coste
- 04** Leyes Heurísticas

La Inversión

El ingeniero de software debe ser consciente del impacto económico que sus decisiones tienen sobre el producto software protegiendo en todo momento la inversión.

“[...] Alguien tiene que pagar por todo esto. El desarrollador de software no puede desconocer el riesgo económico que puede implicar un gran "éxito" técnico. Asegúrese de que está aportando valor de negocio: cumple con los objetivos del negocio y atiende las necesidades del negocio. [...]”

- Kent Beck, Extreme Programming Explained



La Inversión

Las decisiones a tomar en ingeniería del software deben encaminarse a proteger una inversión. El objetivo es hacer esa inversión lo más beneficiosa posible, generando el máximo **valor** cuanto antes, mitigando los **riesgos** que surjan, controlando el **coste** y evitando **endeudarse** demasiado.

- ❑ **Valor.** Es difícil de medir pero muy visible. Valor es lo que queremos. Valor es la funcionalidad deseada (y necesitada) que nos permite ganar más dinero o salvar más vidas.
- ❑ **Coste.** El coste es cuantificable y visible
- ❑ **Deuda.** La deuda es difícil de medir y de observar. Es un coste pospuesto al futuro.
- ❑ **Riesgo.** Al igual que la deuda es difícil de medir y de observar. El riesgo se mide como la probabilidad de fallo (pequeña, mediana, alta) multiplicada por un efecto percibido (pequeño, mediano, grande).

El Valor

Existen algunas métricas que ayudan a medir el valor, como el KPI, Key Performance Indicator. proporcionan una cantidad importante y valorable de información que ayuda en la toma de decisiones.

Un KPI es una cuantificación de un desafío importante de negocio. Por tanto, la naturaleza de un KPI, es un indicador, un valor numérico en el que subyace la importancia de la conexión entre este y los retos de negocio.

Un KPI es una métrica conectada con el valor del negocio:

- ❑ El número de salas de reuniones de la organización es una métrica pero no está conectada con el rendimiento empresarial. Incrementar las salas de reuniones no va a incrementar las ganancias

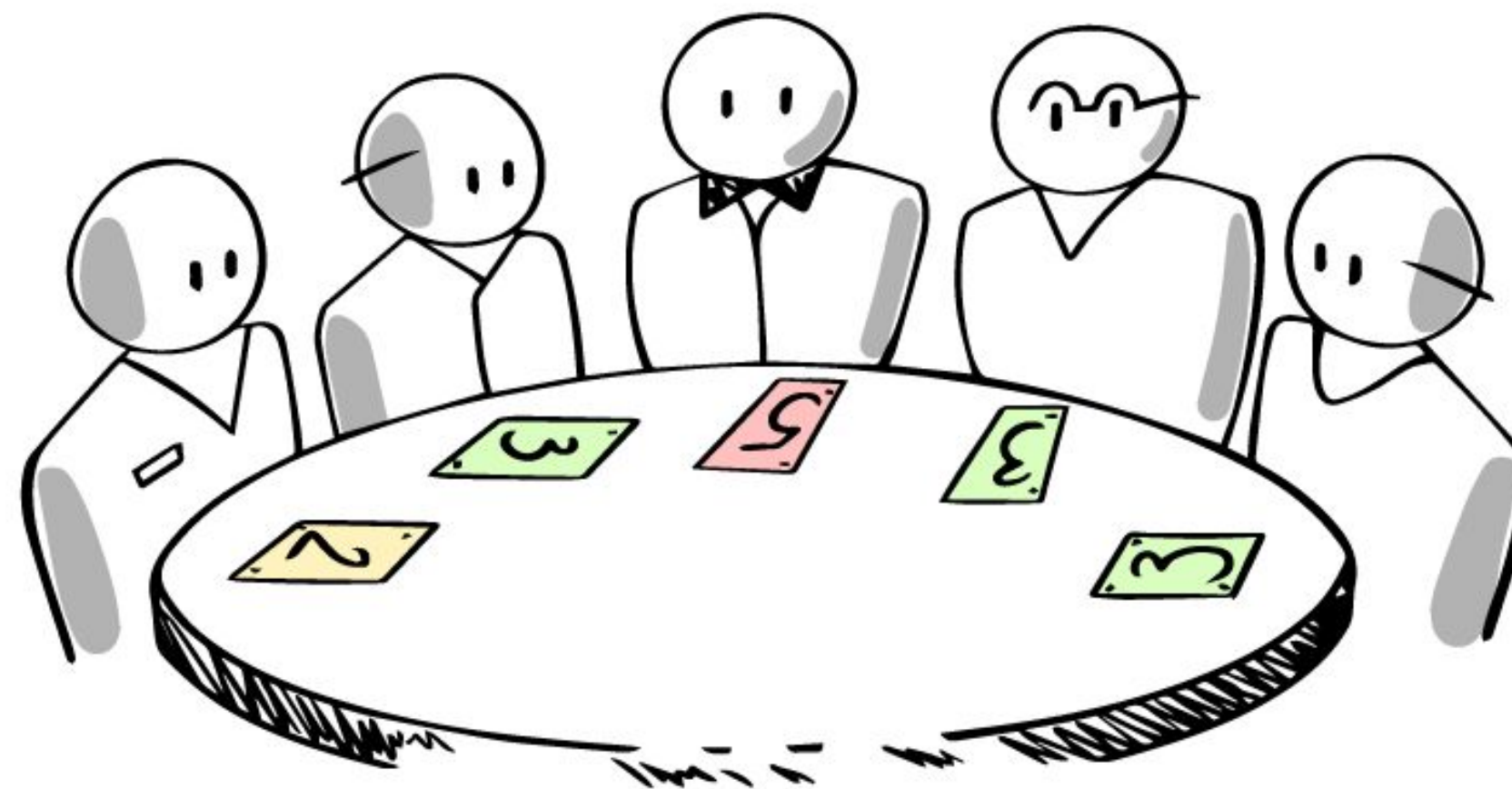
El Coste

En los 80s y 90s se popularizó un modelo de gestión de costes denominado COCOMO. El Modelo Constructivo de Costos (**COCOMO**, por su acrónimo del inglés) es un modelo matemático de base empírica, derivado de la experiencia, utilizado para estimación de costes. Incluye tres submodelos, cada uno ofrece un nivel de detalle y aproximación, cada vez mayor, a medida que avanza el proceso de desarrollo del software: básico, intermedio y detallado. Actualmente apenas es utilizado, en su lugar se utiliza el “juicio del experto” o los costes reales derivados de la construcción de algún otro producto software de naturaleza similar como principal criterio a la hora de estimar el coste. El “juicio del experto” no hace referencia necesariamente a una persona, es habitual que la valoración sea realizada por el equipo técnico de desarrollo.

El Coste

El esfuerzo de desarrollo puede ser medido con “puntos función” o con “puntos historia”:

- Los **Puntos Función** son una métrica que pretende establecer el tamaño en función de la complejidad de los requisitos. Es habitualmente utilizada en la visión predictiva. Es necesario requisitos muy detallados para poder extraer todos los parámetros necesarios para calcular los “puntos función” (existe una fórmula predefinida).
- Los **Puntos Historia** son una métrica para establecer el “tamaño” de una Historia de Usuario en función del esfuerzo. Una HU de dos Puntos Historia se estima que requerirá el doble de esfuerzo que una HU de un único Punto Historia. Los Puntos Historia se obtienen a partir de la propia experiencia del equipo de desarrollo. La estimación de los puntos de historia es realizada por el propio equipo durante el planning meeting, habitualmente a través de una liturgia conocida como “*planning poker*”.

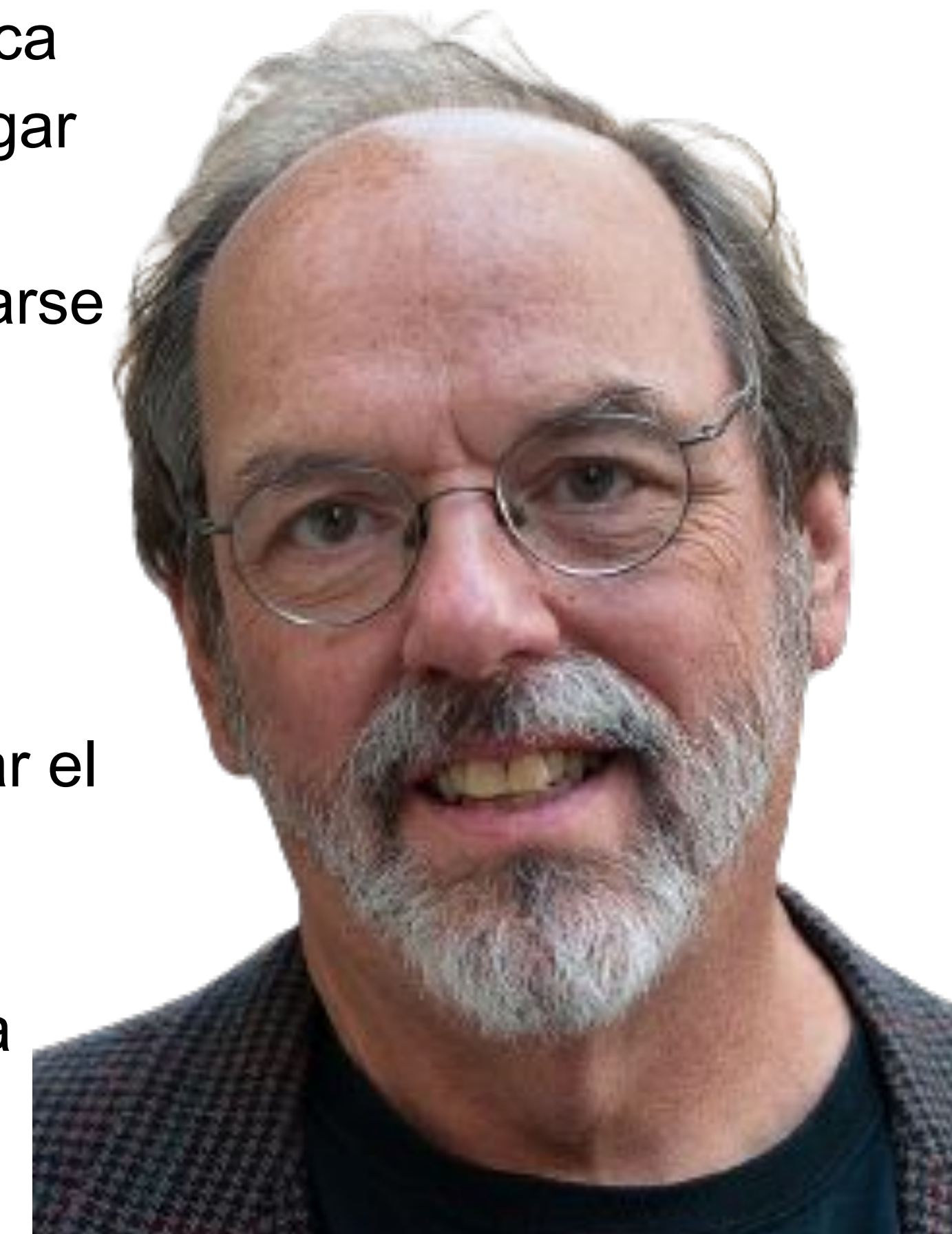


La Deuda

La deuda es el coste futuro. En 1992, Ward Cunningham, firmante del Manifiesto Ágil, introdujo la metáfora de la deuda técnica para explicar a roles no técnicos la importancia de la recodificación sistemática de código heredado.

La refactorización es la modificación en el código del producto que no implica cambio funcional alguno. El objetivo de estas refactorizaciones no es entregar más valor al cliente, sino hacer el software más fácil de mantener y evolucionar reduciendo de esta forma la “deuda técnica” que suele acumularse con el aumento de líneas de código.

La deuda no es negativa en sí misma, la premura por realizar ciertos desarrollos puede llevar a los programadores a desplegar en producción código de baja calidad, código difícil de entender cuya evolución y mantenimiento requiere de mucho esfuerzo. La deuda ha permitido financiar el desarrollo. Esta deuda comenzará a resultar problemática cuando sea necesario modificar el código, por ejemplo si se desea cambiar la funcionalidad ofrecida por el software, es en ese momento cuando la deuda comenzará a cobrarse intereses.



El Riesgo

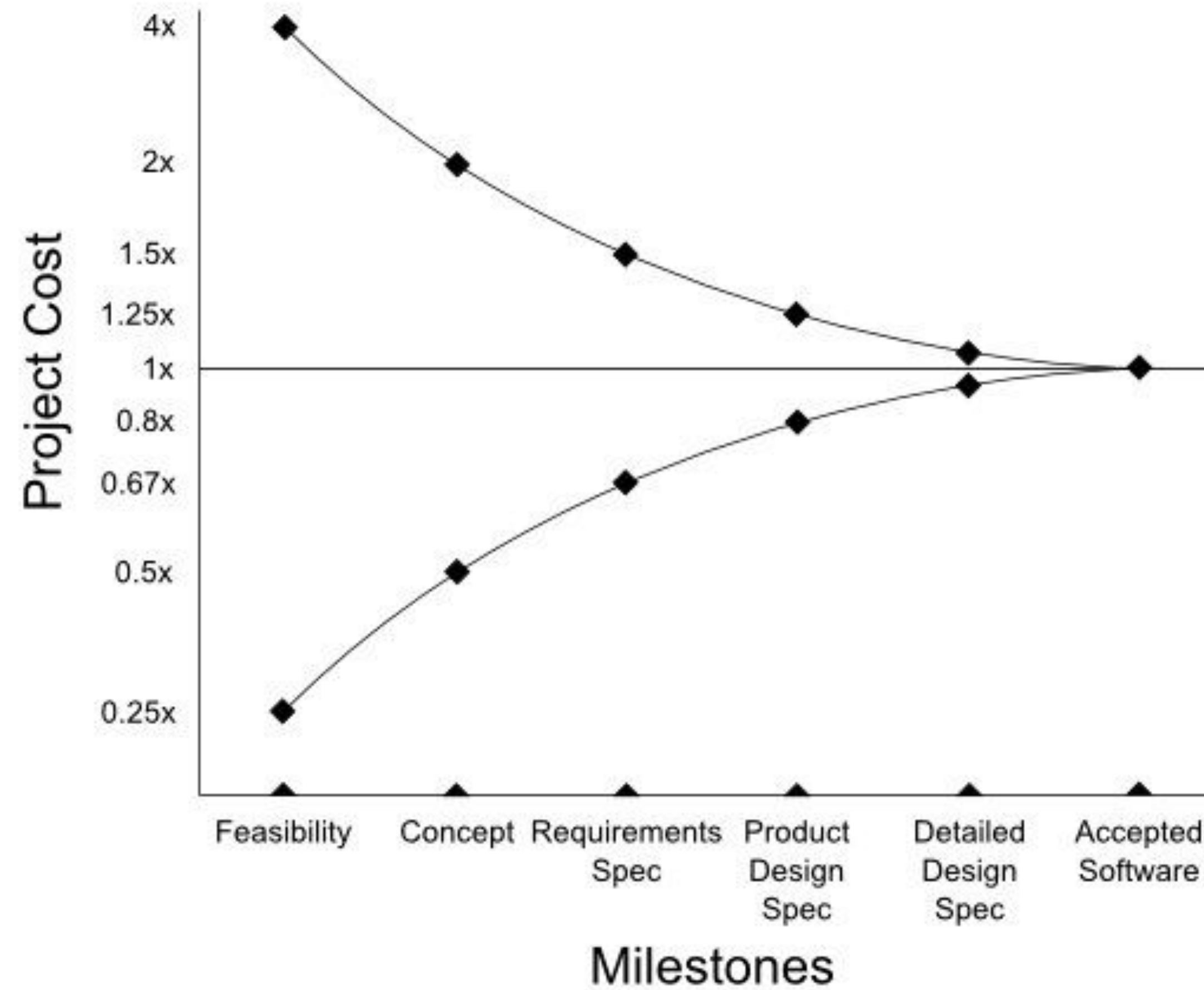
El riesgo es la posibilidad de ser expuesto a una circunstancia adversa o no deseada.

En la visión predictiva es fundamental la gestión activa del riesgo. Es imprescindible identificar los riesgos, analizarlos y definir el plan de acción en cada caso. El análisis del riesgo suele basarse en una matriz donde medir la gravedad del impacto y la probabilidad de que el riesgo se materialice. El plan de riesgos y la matriz impacto/probabilidad suelen documentarse en una hoja de cálculo.

En la visión adaptativa la gestión del riesgo no requiere la generación de tantos artefactos. La propia naturaleza del método de construcción implica la mitigación o reducción de riesgo. La visión adaptativa es segura “por definición”. Las iteraciones cortas y las entregas incrementales del producto permiten detectar y corregir de forma temprana si el producto no satisface las necesidades del cliente. La construcción basada en pruebas automatizadas reduce los errores en el software. Las metodologías ágiles se fundamentan, no en la construcción rápida, sino en la construcción “segura”, minimizando los riesgos.

El Riesgo

Accuracy of Project Cost Estimates versus Milestones

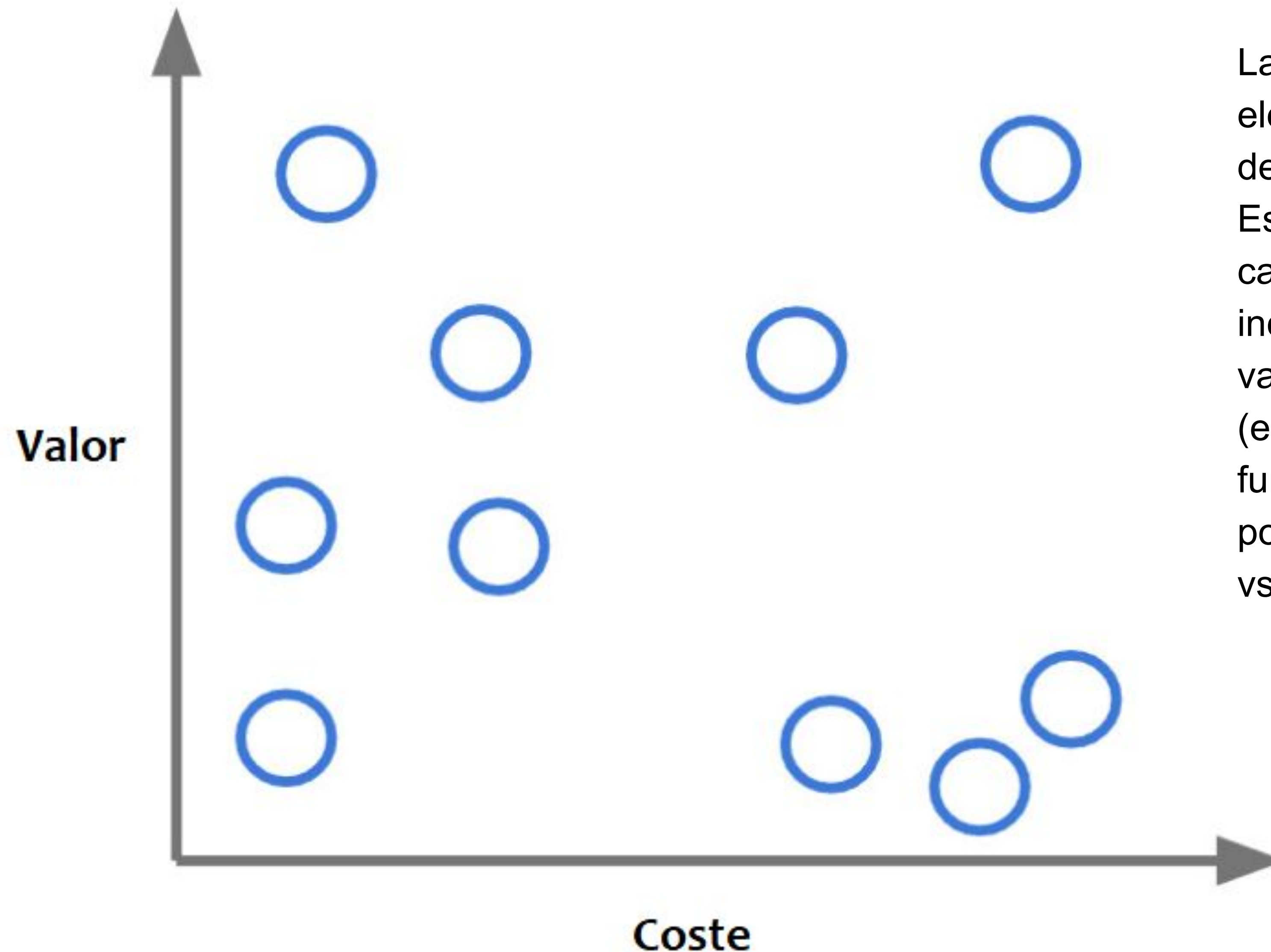


Cono de Incertidumbre

TABLA DE CONTENIDOS

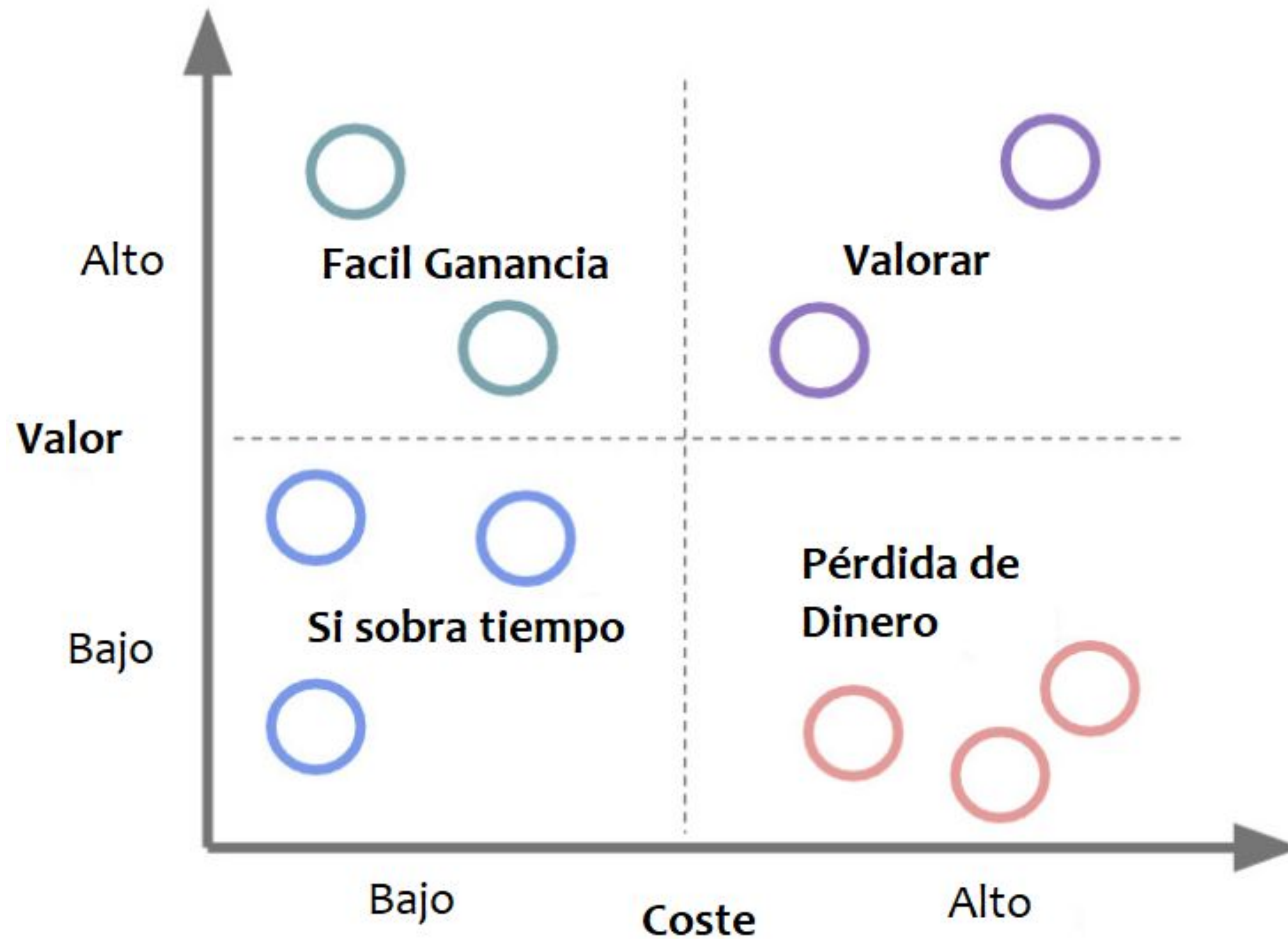
- 01** El triángulo de la gestión
- 02** La inversión
- 03** **Matriz Valor vs Coste**
- 04** Leyes Heurísticas

Beneficio Vs Coste

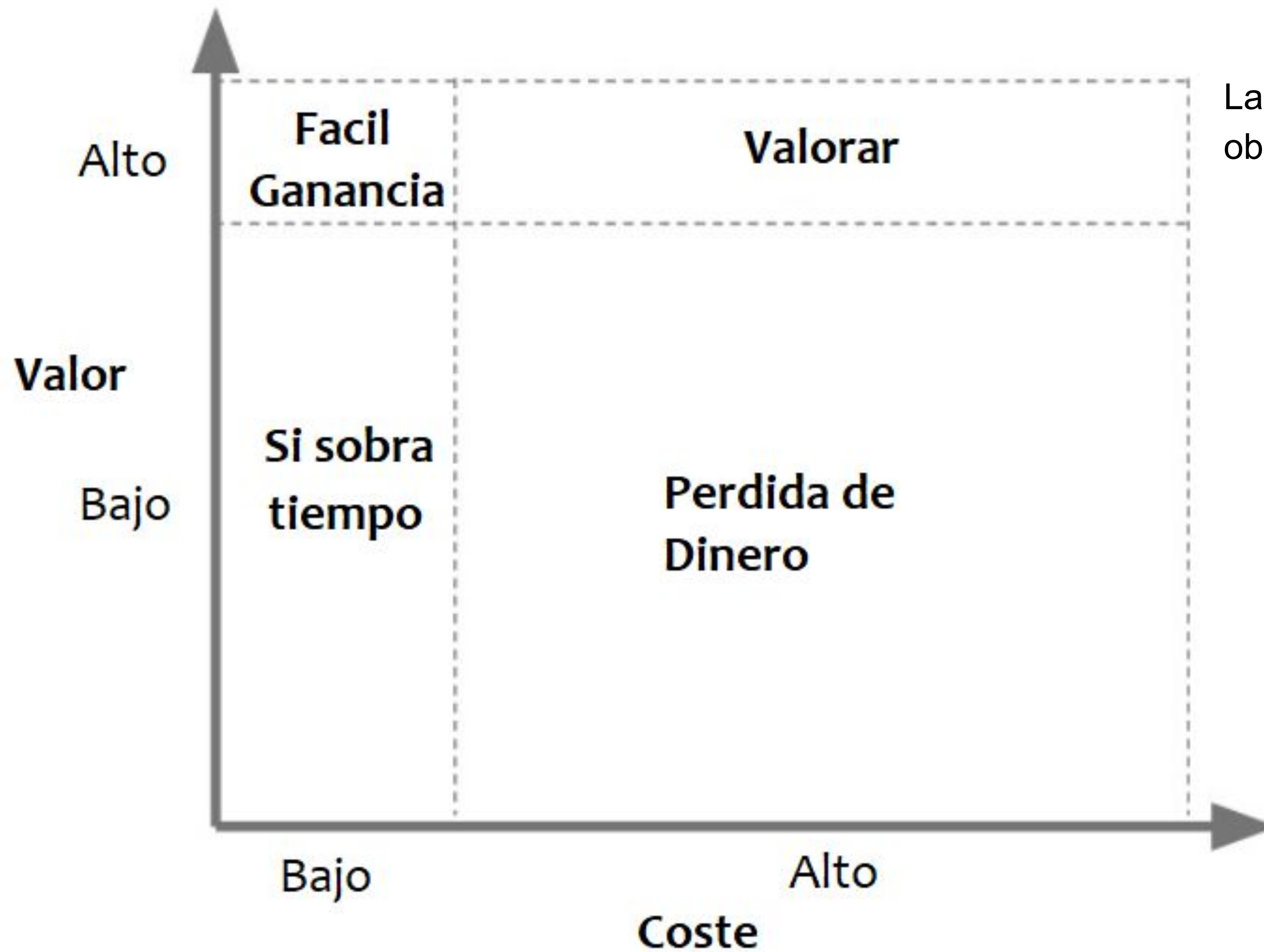


La matriz **Valor vs Coste** es un elemento básico de la priorización de productos, es simple y eficaz. Es suficiente con estimar para cada nueva funcionalidad a incorporar al producto software su valor (impacto) y su coste (esfuerzo). De este modo cada funcionalidad queda representada por un punto en el diagrama valor vs coste.

Beneficio Vs Coste

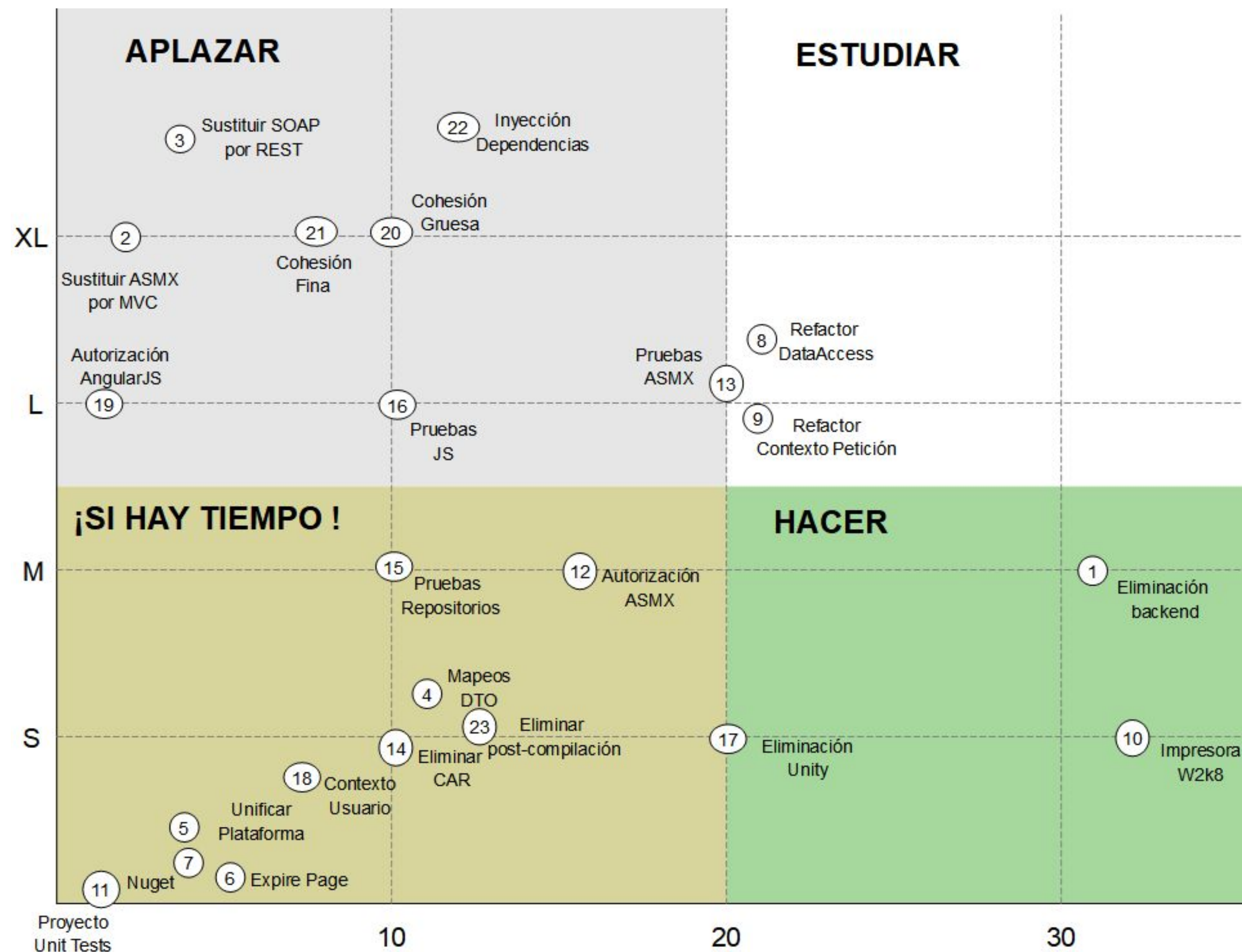


Beneficio Vs Coste



La **falacia de la planificación** obliga a “ajustar” esta matriz.

Beneficio Vs Coste



Es posible utilizar las matrices *beneficio vs coste* para evaluar la conveniencia de ciertas refactorizaciones en el producto software. En el siguiente diagrama se muestra 23 posibles refactorizaciones de un producto software. En el eje de ordenadas se indica el esfuerzo estimado (tallado en S, M, L, XL) que implica implementar la refactorización y el eje de abscisas el beneficio esperado (10, 20, 30) que dicha refactorización entregará al producto al aumentar la velocidad de desarrollo, simplificar los despliegues y el código, facilitar las pruebas, etc.

Beneficio Vs Coste



Estas matrices de valor que ponen en una balanza los beneficios respecto del coste ayudan a mitigar un problema habitual de la industria: la **sobreingeniería**.

La sobreingeniería consiste en dar **soluciones técnicas muy buenas a problemas inexistentes**.

Suele venir acompañada de un incremento en la complejidad del software sin aumentar el valor contraprestado.

La sobreingeniería suele prescindir del punto de vista económico a la hora de tomar decisiones relativas a la construcción del software..

TABLA DE CONTENIDOS

- 01** El triángulo de la gestión
- 02** La inversión
- 03** Matriz Valor vs Coste
- 04** Leyes Heurísticas

LEYES HEURÍSTICAS



Tanto la *ley de brooks* como la *matriz valor vs coste* como la *curva del coste del cambio* como el *radar de deuda técnica* como la *ley de las dos pizzas* como la *ley de Conway* son leyes heurísticas, han sido deducidas a partir del análisis de las mediciones y los datos empíricos (experimentales) obtenidos en la realización de estudios y proyectos.

Estas leyes heurísticas no surgieron de un modelo científico, sino de la experiencia. La formulación de estas leyes o pautas suponen el fundamento cualitativo para la propuesta de metodologías y prácticas utilizadas en la ingeniería del software.